

SPRING CORE AND MAVEN LIBRARY MANAGEMENT SYSTEM

Exercises 1–9: Code and Output

Exercise 1 – Spring Core Library Setup

Description

This exercise introduces the basic Spring Core project structure for a Library Management System. It demonstrates how the Spring container loads objects from an XML configuration file and retrieves the required beans.

Mandatory Requirements

- Create a Maven-based Java project named LibraryManagement.
- Create separate packages for main, repository, and service classes.
- Configure BookRepository and BookService as Spring beans.
- Load applicationContext.xml using ClassPathXmlApplicationContext.
- Display the library welcome message and available books in the console.

Setup Requirements

- JDK 8 or later.
- Eclipse IDE with Maven support.
- Apache Maven configured in Eclipse.
- Spring Context dependency added in pom.xml.
- applicationContext.xml placed inside src/main/resources.

Code

```
// LibraryManagementApplication.java
package com.library.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.repository.BookRepository;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);
        BookRepository repository =
            context.getBean("bookRepository", BookRepository.class);

        service.welcomeUser();
        repository.displayAvailableBooks();

        ((ClassPathXmlApplicationContext) context).close();
    }
}
```

BookRepository.java

```
package com.library.repository;

public class BookRepository {

    public void displayAvailableBooks() {
        System.out.println("\n===== AVAILABLE BOOKS =====");
        System.out.println("1. Java Programming");
        System.out.println("2. Spring Framework");
        System.out.println("3. Database Management Systems");
        System.out.println("4. Python for Data Science");
        System.out.println("5. Machine Learning Basics");
    }
}
```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showLibrary() {
        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("Welcome to the Digital Library!");
        bookRepository.displayAvailableBooks();
    }
}
```

```
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

LibraryManagementApplication.java

```
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            context.getBean("bookService", BookService.class);

        service.showLibrary();
        context.close();
    }
}
```

Output

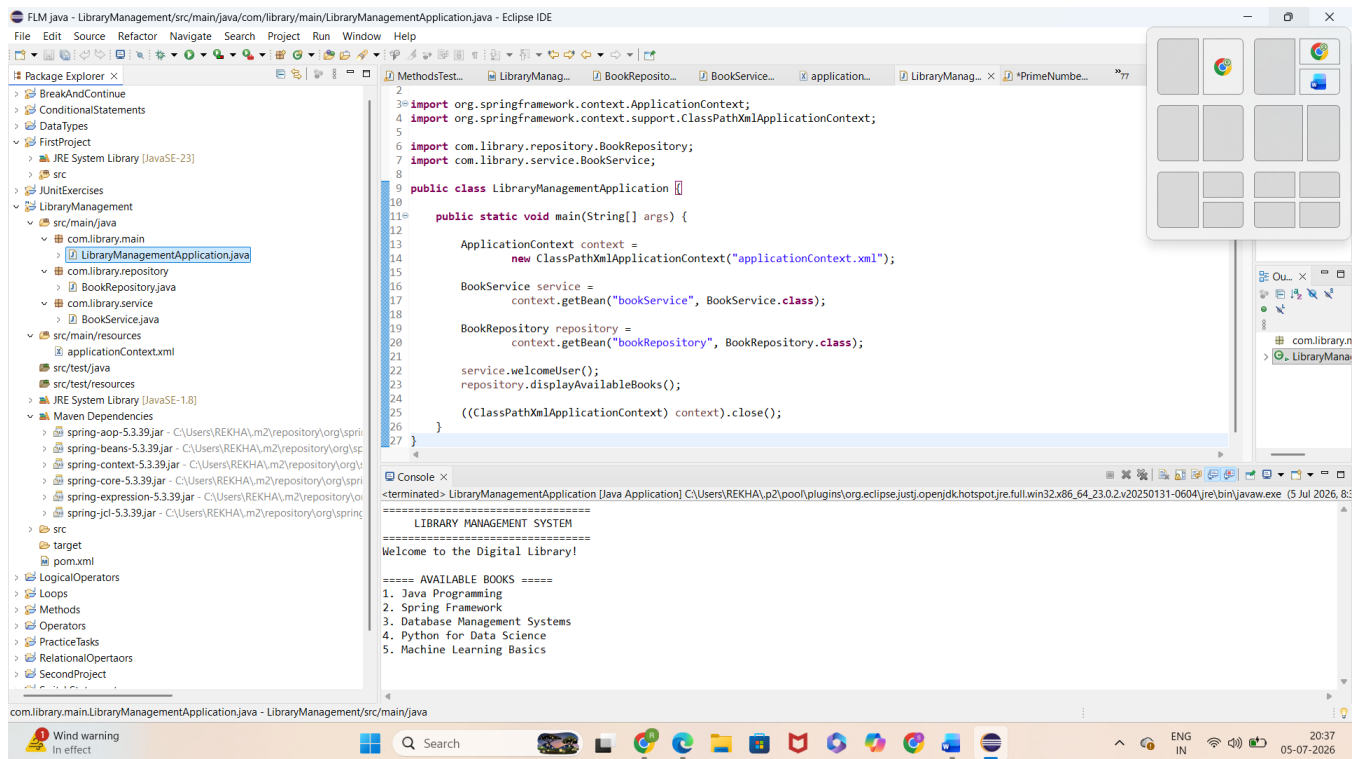


Figure 1: Eclipse console output for Exercise 1

Exercise 2 – Dependency Injection

Description

This exercise demonstrates Dependency Injection in Spring. The BookRepository object is supplied to BookService by the Spring IoC container instead of being created manually inside the service class.

Mandatory Requirements

- Create BookRepository and BookService classes.
- Configure both classes as Spring beans.
- Inject BookRepository into BookService.
- Retrieve BookService from the Spring container.
- Verify dependency injection through the console output.

Setup Requirements

- Completed basic Spring Core project setup.
- Spring Context dependency available.
- Correct bean definitions in applicationContext.xml.
- BookService and BookRepository packages created.
- Project updated and built successfully with Maven.

Code

```
// LibraryManagementApplication.java
package com.library.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
```

```

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);
        service.showLibrary();

        ((ClassPathXmlApplicationContext) context).close();
    }
}

```

BookRepository.java

```

package com.library.repository;

public class BookRepository {

    public void displayAvailableBooks() {
        System.out.println("\n===== AVAILABLE BOOKS =====");
        System.out.println("1. Java Programming");
        System.out.println("2. Spring Framework");
        System.out.println("3. Database Management Systems");
        System.out.println("4. Python for Data Science");
        System.out.println("5. Machine Learning Basics");
    }
}

```

BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showLibrary() {
        System.out.println("=====");
        System.out.println("        LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("Welcome to the Digital Library!");
        bookRepository.displayAvailableBooks();
        System.out.println("\nDependency Injection is working successfully.");
    }
}

```

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

```

```
</beans>
```

LibraryManagementApplication.java

```
package com.library.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            context.getBean("bookService", BookService.class);

        service.showLibrary();

        ((ClassPathXmlApplicationContext) context).close();
    }
}
```

Output

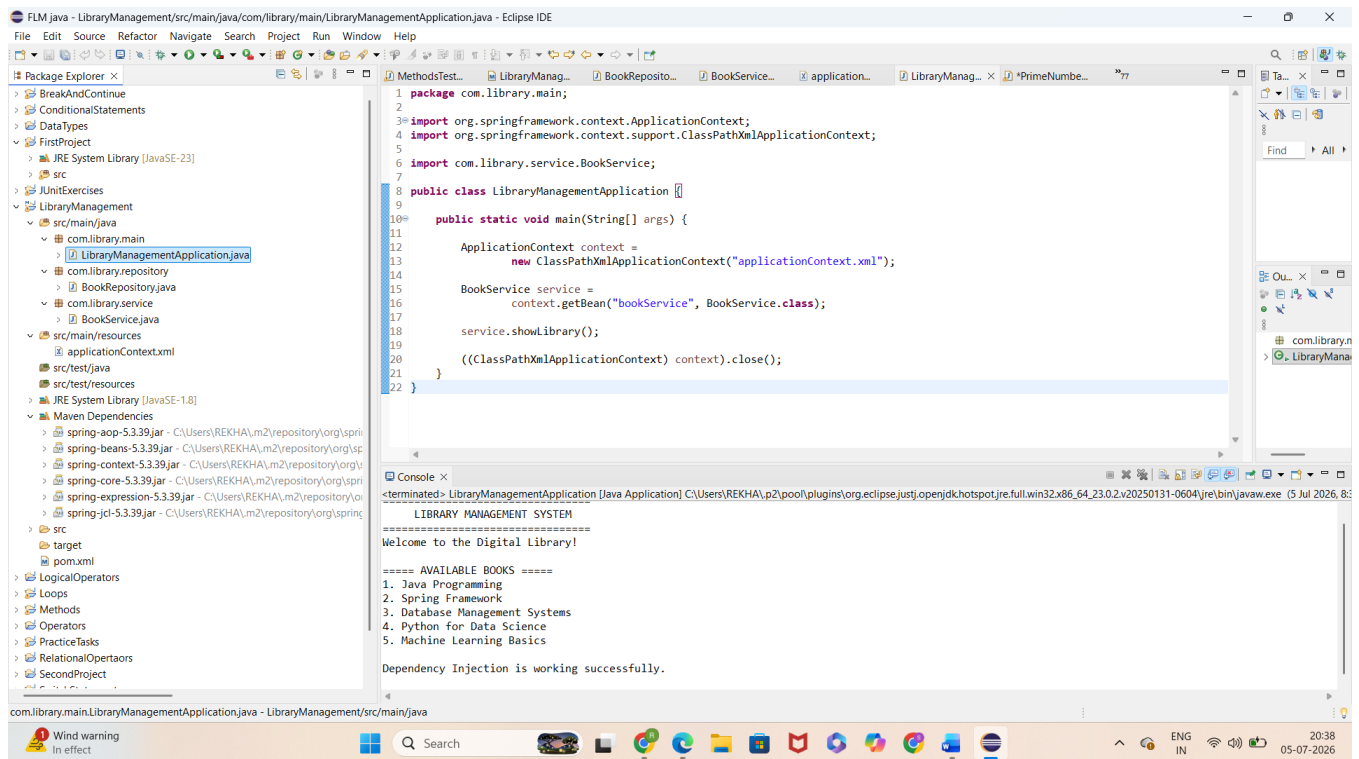


Figure 2: Eclipse console output for Exercise 2

Exercise 3 – Spring AOP Execution Logging

Description

This exercise introduces Aspect-Oriented Programming (AOP) for execution logging. A separate logging aspect measures and displays information about the execution of the library service method.

Mandatory Requirements

- Create a LoggingAspect class.
- Define a pointcut for the BookService method.
- Use AOP advice to capture method execution.
- Display the executed method name and execution time.
- Enable Spring AOP configuration.

Setup Requirements

- Spring Core project from previous exercises.
- Spring AOP and AspectJ Weaver dependencies in pom.xml.
- LoggingAspect class under the aspect package.
- AOP configuration or component scanning enabled.
- Maven dependencies downloaded without errors.

Code

```
// LibraryManagementApplication.java
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
```



```

        BookService service = context.getBean("bookService", BookService.class);
        service.showLibrary();

        context.close();
    }
}

// LoggingAspect.java
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.library.service.BookService.showLibrary(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long end = System.currentTimeMillis();
        System.out.println("\n===== EXECUTION LOG =====");
        System.out.println("Method: showLibrary");
        System.out.println("Execution Time: " + (end - start) + " ms");
        return result;
    }
}

```

BookRepository.java

```

package com.library.repository;

public class BookRepository {

    public void displayAvailableBooks() {
        System.out.println("\n===== AVAILABLE BOOKS =====");
        System.out.println("1. Java Programming");
        System.out.println("2. Spring Framework");
        System.out.println("3. Database Management Systems");
        System.out.println("4. Python for Data Science");
        System.out.println("5. Machine Learning Basics");
    }
}

```

BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showLibrary() {
        System.out.println("=====");
        System.out.println("        LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("Welcome to the Digital Library!");
        bookRepository.displayAvailableBooks();
        System.out.println("\nDependency Injection is working successfully.");
    }
}

```

LoggingAspect.java

```
package com.library.aspect;

import org.aspectj.lang.ProceedingJoinPoint;

public class LoggingAspect {

    public Object logExecutionTime(ProceedingJoinPoint joinPoint)
        throws Throwable {

        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long end = System.currentTimeMillis();

        System.out.println("\n===== EXECUTION LOG =====");
        System.out.println("Method: " + joinPoint.getSignature().getName());
        System.out.println("Execution Time: " + (end - start) + " ms");

        return result;
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
https://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

    <bean id="loggingAspect"
          class="com.library.aspect.LoggingAspect"/>

    <aop:config>
        <aop:aspect ref="loggingAspect">
            <aop:around method="logExecutionTime"
                pointcut="execution(* com.library.service.BookService.showLibrary(..))"/>
        </aop:aspect>
    </aop:config>

</beans>
```

LibraryManagementApplication.java

```
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {
```

```

ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("applicationContext.xml");

BookService service =
    context.getBean("bookService", BookService.class);

service.showLibrary();
context.close();
    }
}

```

Output

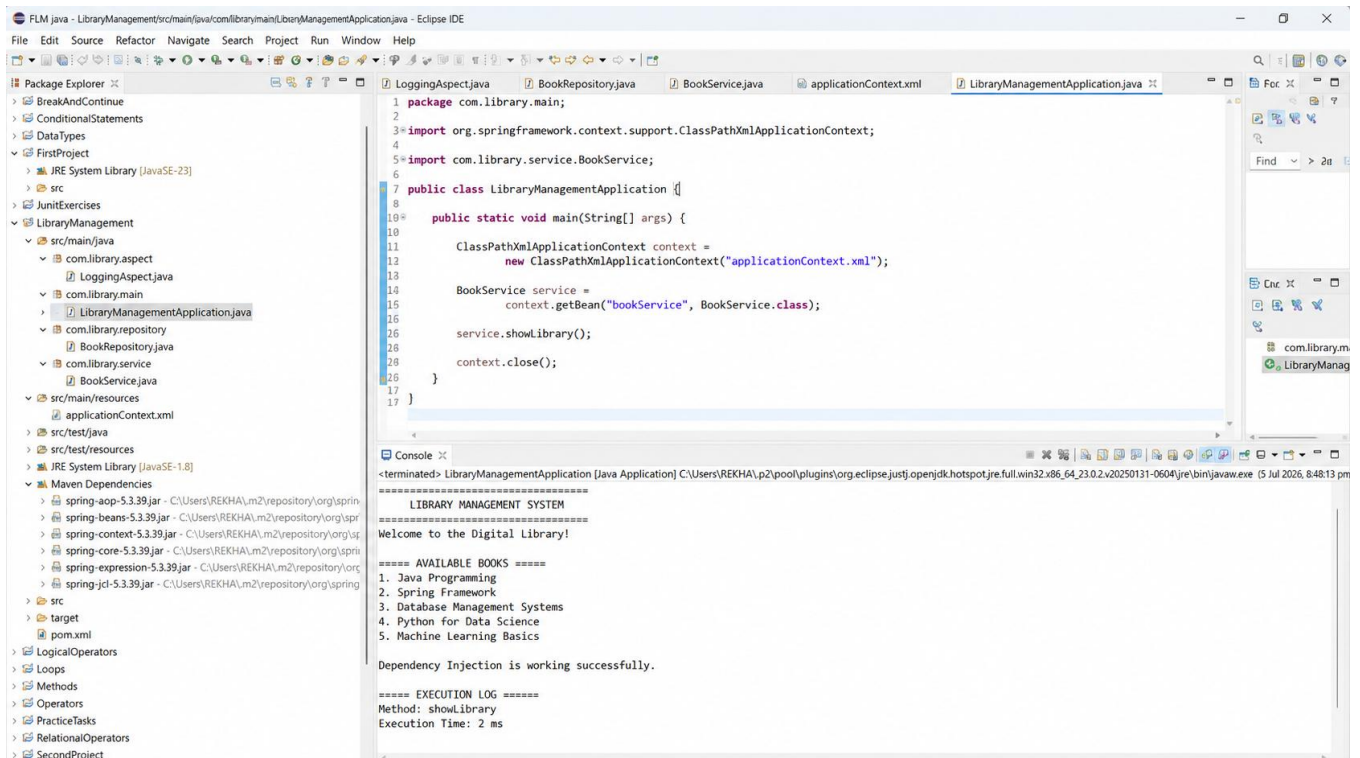


Figure 3: Eclipse console output for Exercise 3

Exercise 4 – Maven Project Configuration

Description

This exercise focuses on Maven project configuration. It verifies the project coordinates, packaging type, Spring dependencies, and Java compiler source and target versions required by the Library Management application.

Mandatory Requirements

- Define Group ID, Artifact ID, version, and packaging in pom.xml.
- Add required Spring dependencies.
- Configure Maven compiler source and target versions.
- Maintain the standard Maven directory structure.
- Verify that the project builds successfully.

Setup Requirements

- JDK 8 or later installed.
- Eclipse IDE with Maven integration.
- Internet connection for downloading Maven dependencies.
- Valid pom.xml in the project root.
- Maven Dependencies visible in the Eclipse Package Explorer.

Code

```
// Exercise 4 - Maven Project Configuration
package com.library.main;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");

        System.out.println("\nExercise 4: Maven Project Configuration");
        System.out.println("\nProject Details:");
        System.out.println("Project Name : LibraryManagement");
        System.out.println("Group Id    : com.library");
        System.out.println("Artifact Id  : LibraryManagement");
        System.out.println("Version     : 0.0.1-SNAPSHOT");
    }
}
```

```

        System.out.println("Packaging      : jar");

        System.out.println("\nSpring Dependencies Configured:");
        System.out.println("1. Spring Context");
        System.out.println("2. Spring AOP");
        System.out.println("3. Spring WebMVC");

        System.out.println("\nMaven Compiler Configuration:");
        System.out.println("Java Source Version : 1.8");
        System.out.println("Java Target Version : 1.8");

        System.out.println("\nMaven project configured successfully.");
        System.out.println("=====");
        System.out.println("      EXERCISE 4 COMPLETED");
        System.out.println("=====");
    }
}

```

pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.39</version>
        </dependency>

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-aop</artifactId>
            <version>5.3.39</version>
        </dependency>

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>5.3.39</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                </configuration>
            </plugin>
        </plugins>
    </build>

```

```

    </build>
</project>

```

LibraryManagementApplication.java

```

package com.library.main;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("        LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");

        System.out.println("\nExercise 4: Maven Project Configuration");

        System.out.println("\nProject Details:");
        System.out.println("Project Name : LibraryManagement");
        System.out.println("Group Id    : com.library");
        System.out.println("Artifact Id : LibraryManagement");
        System.out.println("Version     : 0.0.1-SNAPSHOT");
        System.out.println("Packaging   : jar");

        System.out.println("\nSpring Dependencies Configured:");
        System.out.println("1. Spring Context");
        System.out.println("2. Spring AOP");
        System.out.println("3. Spring WebMVC");

        System.out.println("\nMaven Compiler Configuration:");
        System.out.println("Java Source Version : 1.8");
        System.out.println("Java Target Version : 1.8");

        System.out.println("\nMaven project configured successfully.");
        System.out.println("=====");
        System.out.println("        EXERCISE 4 COMPLETED");
        System.out.println("=====");
    }
}

```

Output

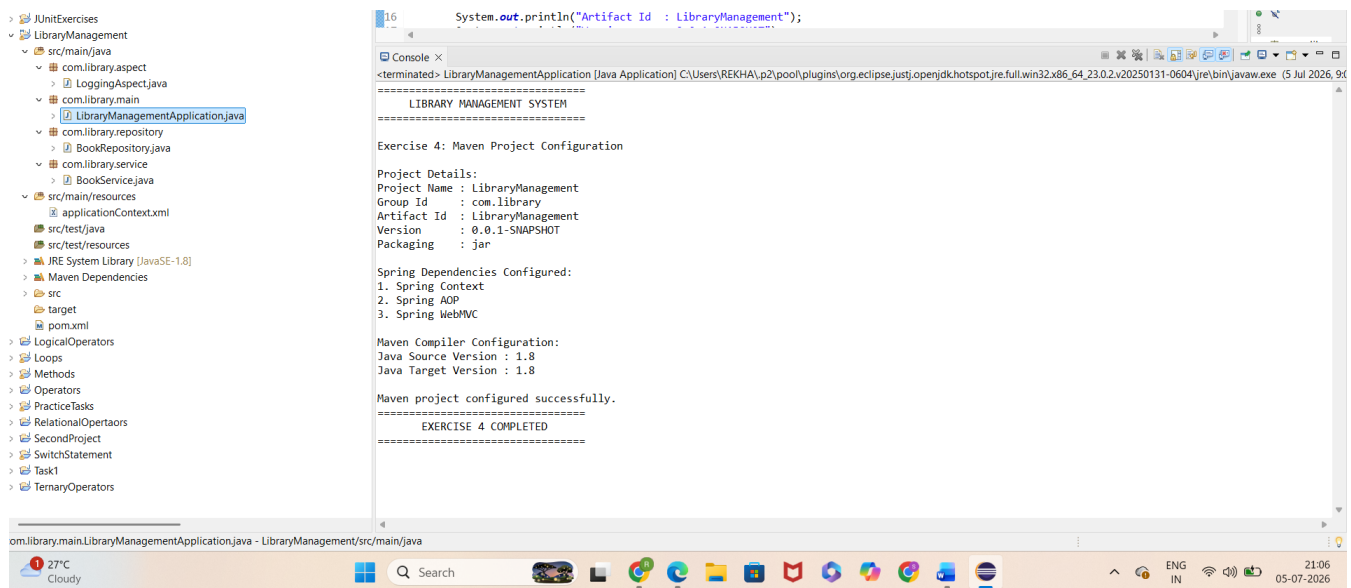


Figure 4: Eclipse console output for Exercise 4

Exercise 5 – Spring IoC Container Configuration

Description

This exercise demonstrates the Spring IoC container. The container creates the BookService and BookRepository beans, manages their lifecycle, and injects the repository dependency into the service.

Mandatory Requirements

- Configure the Spring IoC container.
- Define or discover BookService and BookRepository beans.
- Inject BookRepository into BookService.
- Retrieve BookService using getBean().
- Perform a library search operation and close the context.

Setup Requirements

- Spring Context dependency configured.
- applicationContext.xml available in src/main/resources.
- BookService and BookRepository classes compiled successfully.
- Correct bean IDs used in Java and XML.
- Maven project refreshed and cleaned.

Code

```
// Exercise 5 - Spring IoC Container Configuration
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
```



```

        System.out.println("=====");
        System.out.println("\nExercise 5: Spring IoC Container Configuration");

        System.out.println("\nSpring IoC Container started successfully.");
        System.out.println("BookService bean created by Spring.");
        System.out.println("BookRepository bean created by Spring.");
        System.out.println("BookRepository injected into BookService.");

        BookService service = context.getBean("bookService", BookService.class);
        service.searchBook();

        System.out.println("\nSpring IoC Container is working successfully.");
        System.out.println("=====");
        System.out.println("        EXERCISE 5 COMPLETED");
        System.out.println("=====");
        context.close();
    }
}

```

BookRepository.java

```

package com.library.repository;

public class BookRepository {

    public BookRepository() {
        System.out.println("BookRepository bean created by Spring.");
    }

    public void searchBook() {
        System.out.println("\nLibrary Operation:");
        System.out.println("Searching for book in the library...");
        System.out.println("\nBook found in the library database.");
        System.out.println("Book Name : Spring Framework");
        System.out.println("Author      : Rod Johnson");
        System.out.println("Status      : Available");
    }
}

```

BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public BookService() {
        System.out.println("BookService bean created by Spring.");
    }

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("BookRepository injected into BookService.");
    }

    public void performLibraryOperation() {
        bookRepository.searchBook();
    }
}

```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

LibraryManagementApplication.java

```
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 5: Spring IoC Container Configuration\n");

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("Spring IoC Container started successfully.");

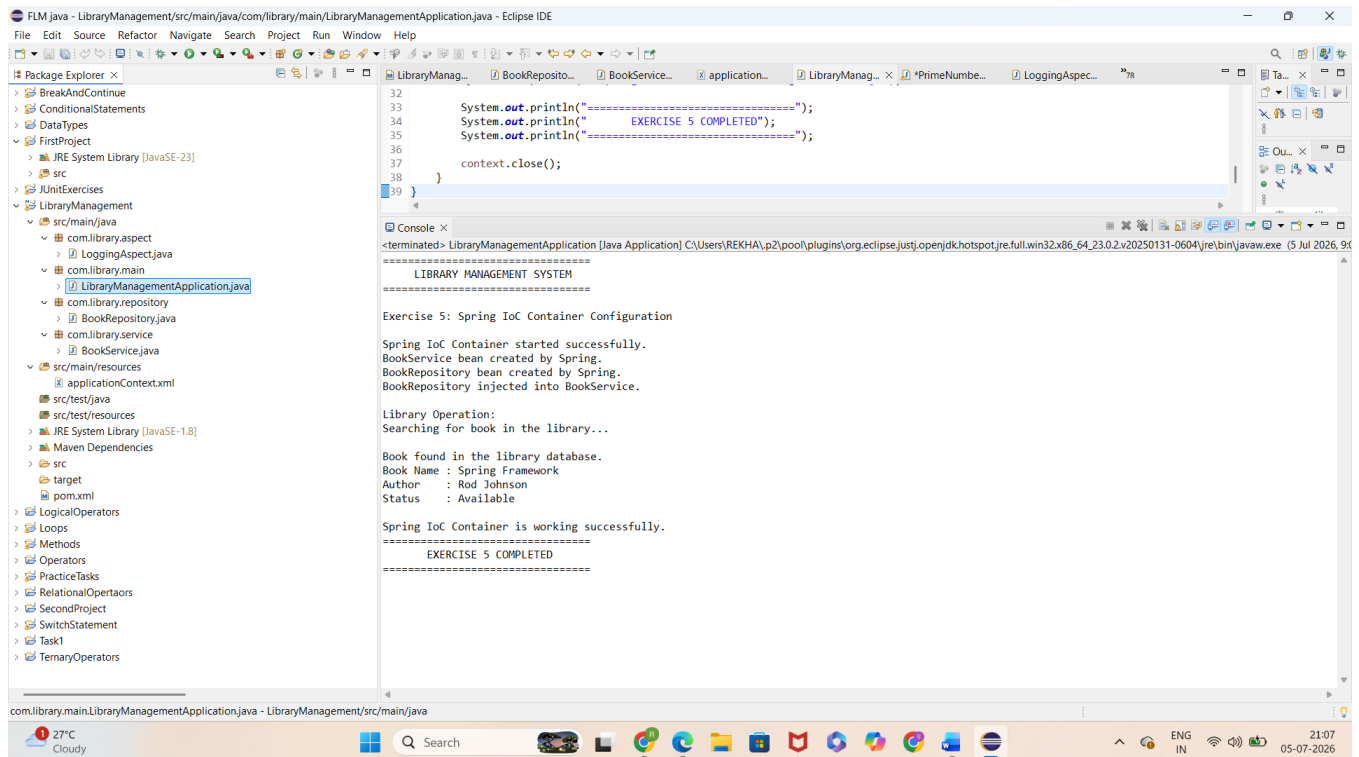
        BookService service =
            context.getBean("bookService", BookService.class);

        service.performLibraryOperation();

        System.out.println("\nSpring IoC Container is working successfully.");
        System.out.println("=====");
        System.out.println("    EXERCISE 5 COMPLETED");
        System.out.println("=====");

        context.close();
    }
}
```

Output



```
com.library.main.LibraryManagementApplication.java - LibraryManagement/src/main/java
File Edit Source Refactor Navigate Search Project Run Window Help
LibraryManag... BookReposito... BookService... application... LibraryManag... *PrimeNumbe... LoggingAspec...
Package Explorer
  BreakAndContinue
  ConditionalStatements
  DataTypes
  FirstProject
  JRE System Library [JavaSE-23]
  src
  UnitExercises
  LibraryManagement
    src/main/java
      com.library.aspect
      LoggingAspect.java
      com.library.main
      LibraryManagementApplication.java
      com.library.repository
      BookRepository.java
      com.library.service
      BookService.java
    src/main/resources
      applicationContext.xml
    src/test/java
    src/test/resources
  JRE System Library [JavaSE-1.8]
  Maven Dependencies
  pom.xml
  LogicalOperators
  Loops
  Methods
  Operators
  PracticeTasks
  RelationalOperators
  SecondProject
  SwitchStatement
  Task1
  TernaryOperators

32
33
34     System.out.println("=====");
35     System.out.println("         EXERCISE 5 COMPLETED");
36     System.out.println("=====");
37     context.close();
38 }
39

Console
<terminated> LibraryManagementApplication [Java Application] C:\Users\REKHA\p2\pool\plugins\org.eclipse.justi.openjdk hotspot\jre.full.win32.x86_64.23.0.2.v20250131-0604\jre\bin\javaw.exe (5 Jul 2026, 94
LIBRARY MANAGEMENT SYSTEM
=====
Exercise 5: Spring IoC Container Configuration
Spring IoC Container started successfully.
BookService bean created by Spring.
BookRepository bean created by Spring.
BookRepository injected into BookService.
Library Operation:
Searching for book in the library...
Book found in the library database.
Book Name : Spring Framework
Author   : Rod Johnson
Status  : Available
Spring IoC Container is working successfully.
=====
EXERCISE 5 COMPLETED
=====
```

Figure 5: Eclipse console output for Exercise 5

Exercise 6 – Annotation-Based Bean Configuration

Description

This exercise replaces manual XML bean declarations with annotation-based configuration. Spring component scanning automatically discovers the service and repository classes.

Mandatory Requirements

- Annotate BookService with @Service.
- Annotate BookRepository with @Repository.
- Enable component scanning for com.library packages.
- Retrieve the discovered BookService bean from the container.
- Display the library catalogue to verify configuration.

Setup Requirements

- Spring Context dependency available.
- Component scanning configured in applicationContext.xml.
- Service and repository classes placed under scanned packages.
- Required Spring annotations imported.
- Project rebuilt after annotation changes.

Code

```
// Exercise 6 - Annotation-Based Bean Configuration
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 6: Annotation-Based Bean Configuration");
    }
}
```

```

        System.out.println("\nSpring component scanning started successfully.");
        System.out.println("BookService bean created using @Service.");
        System.out.println("BookRepository bean created using @Repository.");

        BookService service = context.getBean("bookService", BookService.class);
        service.showCatalogue();

        System.out.println("\nAnnotation-based configuration is working successfully.");
        System.out.println("=====");
        System.out.println("        EXERCISE 6 COMPLETED");
        System.out.println("=====");
        context.close();
    }
}

```

BookRepository.java

```

package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {

    public BookRepository() {
        System.out.println("BookRepository bean created using @Repository.");
    }

    public void displayCatalogue() {
        System.out.println("\n===== LIBRARY CATALOGUE =====");
        System.out.println("1. Java Programming      - Available");
        System.out.println("2. Spring Framework      - Available");
        System.out.println("3. Machine Learning      - Issued");
        System.out.println("4. Database Management    - Available");
        System.out.println("5. Python for Data Science - Available");
    }
}

```

BookService.java

```

package com.library.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.library.repository.BookRepository;

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public BookService() {
        System.out.println("BookService bean created using @Service.");
    }
}

```

```

    public void showCatalogue() {
        bookRepository.displayCatalogue();
    }
}

```

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.library"/>

</beans>

```

LibraryManagementApplication.java

```

package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 6: Annotation-Based Bean Configuration\n");

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        System.out.println("Spring component scanning started successfully.");

        BookService service = context.getBean(BookService.class);

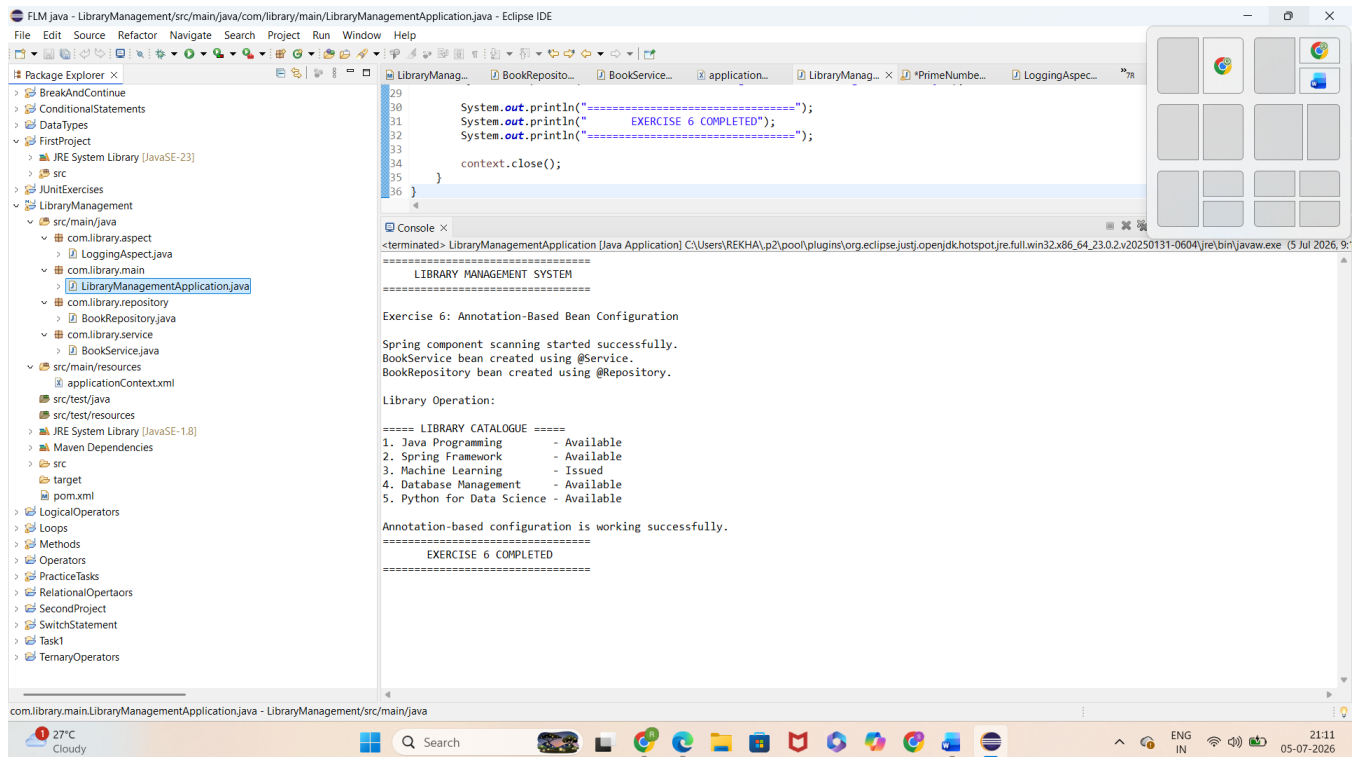
        System.out.println("\nLibrary Operation:");
        service.showCatalogue();

        System.out.println("\nAnnotation-based configuration is working successfully.");
        System.out.println("=====");
        System.out.println("    EXERCISE 6 COMPLETED");
        System.out.println("=====");

        context.close();
    }
}

```

Output



```
com.library.main.LibraryManagementApplication.java - LibraryManagement/src/main/java
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
  BreakAndContinue
  ConditionalStatements
  DataTypes
  FirstProject
    JRE System Library [JavaSE-23]
    src
  JUnitExercises
  LibraryManagement
    src/main/java
      com.library.aspect
      LoggingAspect.java
      com.library.main
      LibraryManagementApplication.java
      com.library.repository
      BookRepository.java
      com.library.service
      BookService.java
    src/main/resources
      applicationContext.xml
    src/test/java
    src/test/resources
  JRE System Library [JavaSE-1.8]
  Maven Dependencies
  target
  pom.xml
  LogicalOperators
  Loops
  Methods
  Operators
  PracticeTasks
  RelationalOperators
  SecondProject
  SwitchStatement
  Task1
  TernaryOperators

LibraryManagementApplication.java
29     System.out.println("=====");
30
31     System.out.println("      EXERCISE 6 COMPLETED");
32     System.out.println("=====");
33
34     context.close();
35 }
36

Console
<terminated> LibraryManagementApplication [Java Application] C:\Users\REKHA\p2\pool\plugins\org.eclipse.justi.openjdk hotspot\jre.full.win32.x86_64.23.0.2.v20250131-0604\jre\bin\javaw.exe (5 Jul 2026, 9:
=====
LIBRARY MANAGEMENT SYSTEM
=====

Exercise 6: Annotation-Based Bean Configuration

Spring component scanning started successfully.
BookService bean created using @Service.
BookRepository bean created using @Repository.

Library Operation:

===== LIBRARY CATALOGUE =====
1. Java Programming - Available
2. Spring Framework - Available
3. Machine Learning - Issued
4. Database Management - Available
5. Python for Data Science - Available

Annotation-based configuration is working successfully.

=====
EXERCISE 6 COMPLETED
=====
```

Figure 6: Eclipse console output for Exercise 6

Exercise 7 – Constructor and Setter Injection

Description

This exercise demonstrates two common dependency injection techniques: constructor injection and setter injection. It shows how Spring can supply required collaborators and configurable values to a service object.

Mandatory Requirements

- Configure constructor injection for BookRepository.
- Configure setter injection for LibraryRepository or a library property.
- Create the required constructor and setter methods.
- Define matching bean references or properties.
- Verify both injection methods through console output.

Setup Requirements

- Spring IoC container working correctly.
- Required repository classes created.
- Constructor and setter methods available in BookService.
- Bean configuration updated for both injection types.
- No duplicate or conflicting bean definitions.

Code

```
// Exercise 7 - Constructor and Setter Injection
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);
    }
}
```



```

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 7: Constructor and Setter Injection");

        System.out.println("\nConstructor Injection:");
        System.out.println("BookRepository injected into BookService.");
        System.out.println("\nSetter Injection:");
        System.out.println("LibraryRepository injected into BookService.");

        service.showLibraryDetails();

        System.out.println("\nBoth injection methods are working successfully.");
        System.out.println("=====");
        System.out.println("    EXERCISE 7 COMPLETED");
        System.out.println("=====");
        context.close();
    }
}

```

BookRepository.java

```

package com.library.repository;

public class BookRepository {

    public BookRepository() {
        System.out.println("BookRepository injected into BookService.");
    }

    public void displayAvailableBooks() {
        System.out.println("\n===== AVAILABLE BOOKS =====");
        System.out.println("1. Java Programming");
        System.out.println("2. Spring Framework");
        System.out.println("3. Database Management Systems");
        System.out.println("4. Python for Data Science");
        System.out.println("5. Machine Learning Basics");
    }
}

```

LibraryRepository.java

```

package com.library.repository;

public class LibraryRepository {

    private String libraryName;

    public void setLibraryName(String libraryName) {
        this.libraryName = libraryName;
    }

    public String getLibraryName() {
        return libraryName;
    }
}

```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;
import com.library.repository.LibraryRepository;

public class BookService {

    private BookRepository bookRepository;
    private LibraryRepository libraryRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void setLibraryRepository(LibraryRepository libraryRepository) {
        this.libraryRepository = libraryRepository;
        System.out.println("LibraryRepository injected into BookService.");
    }

    public void showLibrary() {
        System.out.println("\nLibrary Operation:");
        System.out.println("Library Name : " + libraryRepository.getLibraryName());
        bookRepository.displayAvailableBooks();
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="libraryRepository"
          class="com.library.repository.LibraryRepository">
        <property name="libraryName" value="Digital Central Library"/>
    </bean>

    <bean id="bookService"
          class="com.library.service.BookService">
        <constructor-arg ref="bookRepository"/>
        <property name="libraryRepository" ref="libraryRepository"/>
    </bean>

</beans>
```

LibraryManagementApplication.java

```
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
```

```

public static void main(String[] args) {

    ClassPathXmlApplicationContext context =
        new ClassPathXmlApplicationContext("applicationContext.xml");

    BookService service =
        context.getBean("bookService", BookService.class);

    System.out.println("=====");
    System.out.println("        LIBRARY MANAGEMENT SYSTEM");
    System.out.println("=====");
    System.out.println("\nExercise 7: Constructor and Setter Injection\n");

    System.out.println("Constructor Injection:");
    System.out.println("BookRepository injected into BookService.");

    System.out.println("\nSetter Injection:");
    System.out.println("LibraryRepository injected into BookService.");

    service.showLibrary();

    System.out.println("\nBoth injection methods are working successfully.");
    System.out.println("=====");
    System.out.println("        EXERCISE 7 COMPLETED");
    System.out.println("=====");

    context.close();
}
}

```

Output

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, with the 'LibraryManagementApplication.java' file selected. The main editor window shows the source code of the application. The Console window at the bottom displays the output of the application, which matches the code shown in the previous block. The output includes the library management system header, the exercise title, the constructor and setter injection details, the library operation, and the list of available books.

```

<terminated> LibraryManagementApplication [Java Application] C:\Users\REKHA\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_23.0.2.v20250131-0604\jre\bin\javaw.exe (6 Jul 2026, 3:30)

=====
LIBRARY MANAGEMENT SYSTEM
=====

Exercise 7: Constructor and Setter Injection

Constructor Injection:
BookRepository injected into BookService.

Setter Injection:
LibraryRepository injected into BookService.

Library Operation:
Library Name : Digital Central Library

===== AVAILABLE BOOKS =====
1. Java Programming
2. Spring Framework
3. Database Management Systems
4. Python for Data Science
5. Machine Learning Basics

Both injection methods are working successfully.
=====
EXERCISE 7 COMPLETED
=====

```

Figure 7: Eclipse console output for Exercise 7

Exercise 8 – Basic AOP with Spring

Description

This exercise demonstrates basic Spring AOP using before and after advice. Logging messages are executed automatically around the book issue operation without adding logging code directly to the business method.

Mandatory Requirements

- Create a LoggingAspect class.
- Add @Before advice for the issueBook() method.
- Add @After advice for the same method.
- Enable AspectJ auto-proxy support.
- Run the book issue operation and verify both AOP logs.

Setup Requirements

- Spring AOP dependency added.
- AspectJ Weaver dependency added.
- Aspect class discovered by Spring.
- AOP proxy support enabled.
- BookService issueBook() method available and public.

Code

```
// Exercise 8 - Basic AOP with Spring
package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);
```

```

        System.out.println("=====");
        System.out.println("      LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 8: Basic AOP with Spring");
        System.out.println("\nStarting Book Issue Operation:");

        service.issueBook();

        System.out.println("\nSpring AOP is working successfully.");
        System.out.println("=====");
        System.out.println("      EXERCISE 8 COMPLETED");
        System.out.println("=====");
        context.close();
    }
}

// LoggingAspect.java
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.library.service.BookService.issueBook(..))")
    public void beforeLog() {
        System.out.println("\n[AOP BEFORE LOG]");
        System.out.println("BookService method execution started.");
    }

    @After("execution(* com.library.service.BookService.issueBook(..))")
    public void afterLog() {
        System.out.println("\n[AOP AFTER LOG]");
        System.out.println("BookService method execution completed.");
    }
}

```

BookService.java

```

package com.library.service;

public class BookService {

    public void issueBook() {
        System.out.println("\nProcessing library request...");
        System.out.println("Book Name : Spring Framework");
        System.out.println("Member ID : LIB101");
        System.out.println("Status      : Book issued successfully");
    }
}

```

LoggingAspect.java

```

package com.library.aspect;

import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect
public class LoggingAspect {

```

```

@Before("execution(* com.library.service.BookService.issueBook(..)")
public void beforeLog() {
    System.out.println("\n[AOP BEFORE LOG]");
    System.out.println("BookService method execution started.");
}

@After("execution(* com.library.service.BookService.issueBook(..)")
public void afterLog() {
    System.out.println("\n[AOP AFTER LOG]");
    System.out.println("BookService method execution completed.");
}
}

```

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
https://www.springframework.org/schema/aop/spring-aop.xsd">

    <aop:aspectj-autoproxy/>

    <bean id="bookService"
          class="com.library.service.BookService"/>

    <bean id="loggingAspect"
          class="com.library.aspect.LoggingAspect"/>

</beans>

```

LibraryManagementApplication.java

```

package com.library.main;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            context.getBean("bookService", BookService.class);

        System.out.println("=====");
        System.out.println("    LIBRARY MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("\nExercise 8: Basic AOP with Spring\n");

        System.out.println("Starting Book Issue Operation:");
        service.issueBook();

        System.out.println("\nSpring AOP is working successfully.");
    }
}

```

```

        System.out.println("=====");
        System.out.println("            EXERCISE 8 COMPLETED");
        System.out.println("=====");

        context.close();
    }
}

```

Output

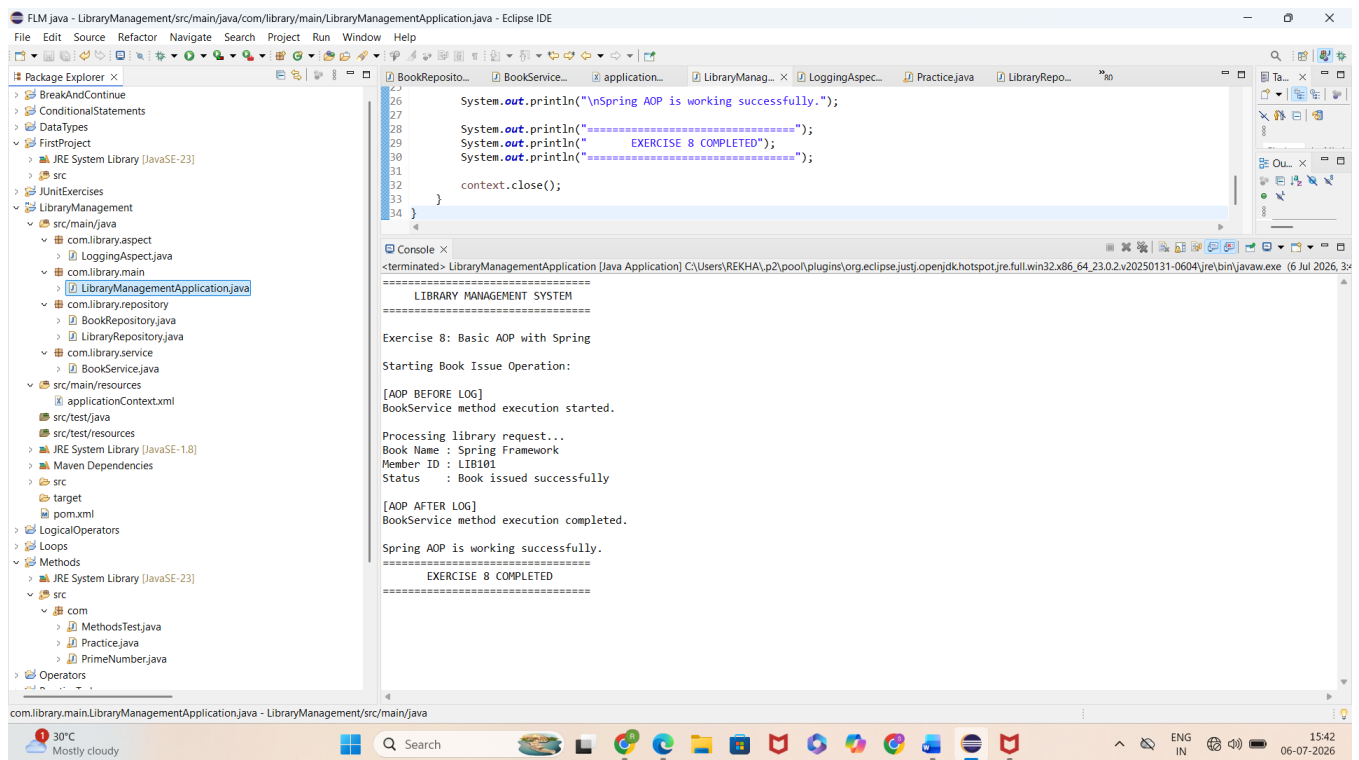


Figure 8: Eclipse console output for Exercise 8

Exercise 9 – Spring Boot Library Management Application

Description

This exercise converts the library project into a Spring Boot application. It uses auto-configuration, Spring Data JPA, an entity class, repository interface, and REST controller to build a web-based Library Management backend.

Mandatory Requirements

- Create a Spring Boot Maven project.
- Add Spring Web and Spring Data JPA dependencies.
- Create the Book entity with an ID field.
- Create BookRepository extending JpaRepository.
- Create a REST controller for book operations.
- Run the application using SpringApplication.run().

Setup Requirements

- JDK 17 recommended for the shown Spring Boot version.
- Eclipse IDE or Spring Tool Suite.
- Maven configured and dependencies downloaded.
- Database configuration added in application.properties when persistence is used.
- Required packages for controller, entity, and repository created.

Code

```
// Exercise 9 - Spring Boot Library Management Application
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

```
// Book.java
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
    // getters and setters
}

// BookRepository.java
public interface BookRepository extends JpaRepository<Book, Long> {
}

// BookController.java
@RestController
@RequestMapping("/books")
public class BookController {
    private final BookRepository repository;

    public BookController(BookRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Book> getAllBooks() {
        return repository.findAll();
    }
}
```

Book.java

```
package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;

    public Book() {
    }

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public Long getId() {
```

```

        return id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }
}

```

BookRepository.java

```

package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.library.entity.Book;

public interface BookRepository extends JpaRepository<Book, Long> {
}

```

BookController.java

```

package com.library.controller;

import java.util.List;
import org.springframework.web.bind.annotation.*;
import com.library.entity.Book;
import com.library.repository.BookRepository;

@RestController
@RequestMapping("/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }
}

```

LibraryManagementApplication.java

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

application.properties

```
spring.application.name=library-management-springboot
server.port=8080

spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.6</version>
        <relativePath/>
    </parent>

    <groupId>com.library</groupId>
    <artifactId>library-management-springboot</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
    </dependencies>
</project>
```

```

<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Output

```

Boot - library-management-springboot/src/main/java/com/library/LibraryManagementApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
  library-management-springboot [boot]
    src/main/java
      com.library
        controller
          BookController.java
        entity
          Book.java
        repository
          BookRepository.java
          LibraryManagementApplication.java
    src/main/resources
      static
      templates
      application.properties
    src/test/java
  JRE System Library [JavaSE-17]
  Maven Dependencies
    target
    HELP.md
    mvnw
    mvnw.cmd
    pom.xml

LibraryManagementApplication.java
1 package com.library;
2
3 import org.springframework.boot.SpringApplication;
4
5 @SpringBootApplication
6 public class LibraryManagementApplication {
7
8     public static void main(String[] args) {
9         SpringApplication.run(LibraryManagementApplication.class, args);
10    }
11 }

Console
<terminated> LibraryManagementApplication (Java Application) C:\Program Files\Java\jdk-17.0.10\bin\javaw.exe (06-Jul-2026, 4:12:25 pm - 4:12:32 pm) [pid: 13456]

Spring Boot 3.2.6
2026-07-06T16:12:25.816+05:30 INFO 13456 --- [main] c.l.LibraryManagementApplication : Starting LibraryManagementApplication
2026-07-06T16:12:25.819+05:30 INFO 13456 --- [main] c.l.LibraryManagementApplication : No active profile set, falling back to 1
2026-07-06T16:12:26.421+05:30 INFO 13456 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories
2026-07-06T16:12:26.486+05:30 INFO 13456 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning
2026-07-06T16:12:27.593+05:30 INFO 13456 --- [main] o.s.b.v.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-06T16:12:27.606+05:30 INFO 13456 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-06T16:12:27.607+05:30 INFO 13456 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.18]
2026-07-06T16:12:27.799+05:30 INFO 13456 --- [main] o.a.c.c.c.[Tomcat].[localhost].[] : Initializing Spring embedded WebApplicationContext
2026-07-06T16:12:27.710+05:30 INFO 13456 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
2026-07-06T16:12:27.886+05:30 INFO 13456 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting..
2026-07-06T16:12:28.069+05:30 INFO 13456 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-07-06T16:12:28.087+05:30 INFO 13456 --- [main] o.hibernate.jpa.internal.util.LogHelper : HH0000204: Processing PersistenceUnitInfo [name: default]
2026-07-06T16:12:28.153+05:30 INFO 13456 --- [main] org.hibernate.Version : HH0000412: Hibernate ORM core version 6.4.4.Final
2026-07-06T16:12:28.329+05:30 INFO 13456 --- [main] o.h.c.internal.RegionFactoryInitiator : HH0000026: Second-level cache disabled

```

Figure 9: Eclipse console output for Exercise 9